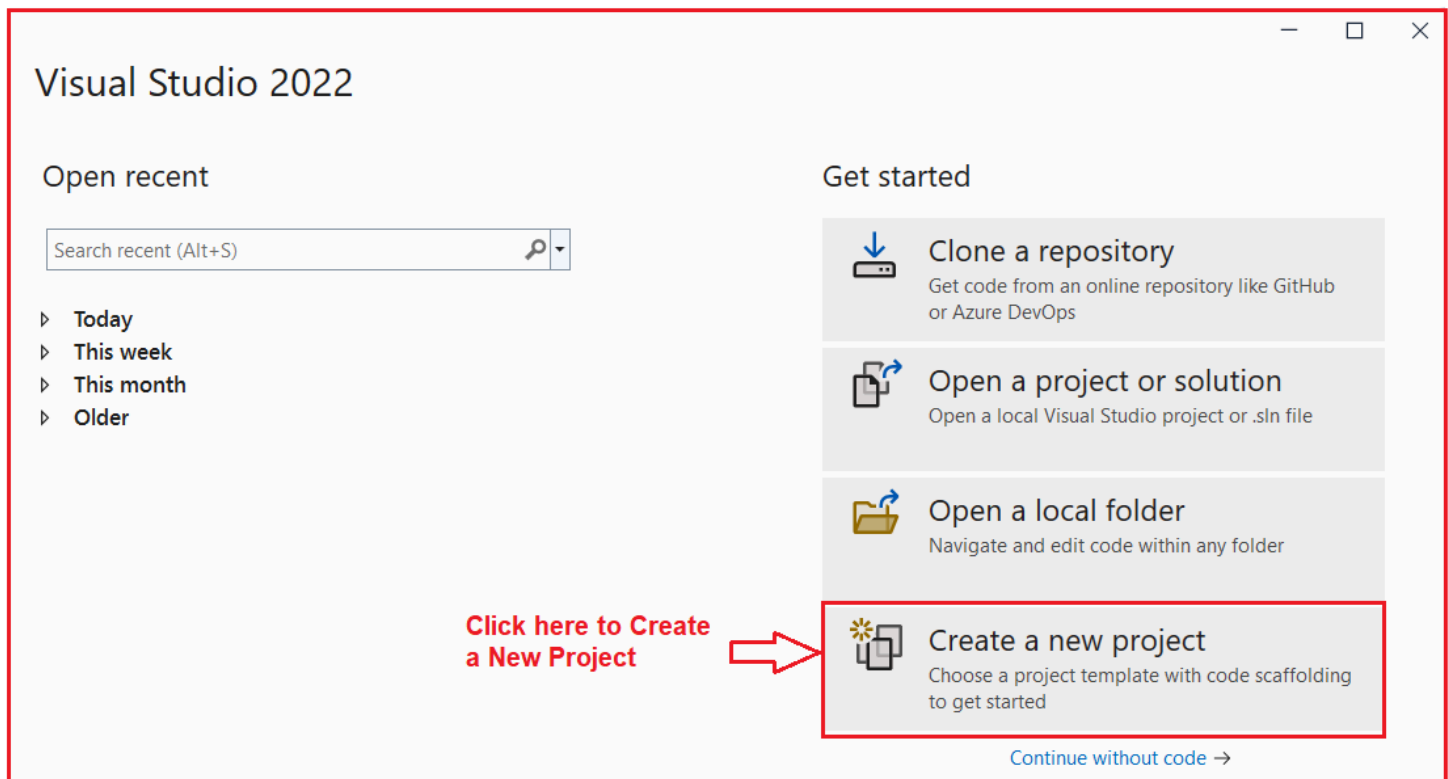


How to Install Entity Framework Core in .NET Core Application

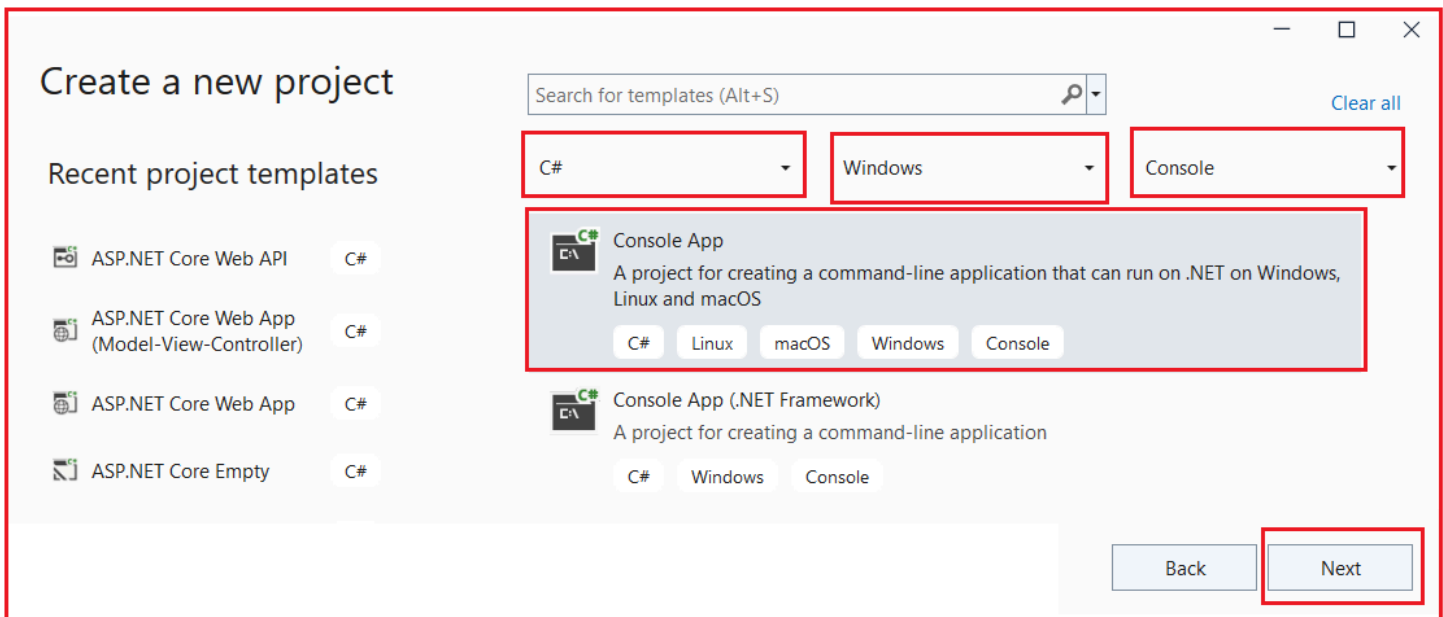
In this article, I will show you **How to Install and use Entity Framework Core in .NET Core Console Application** using Visual Studio. Once we understand the Entity Framework Core Basic Concepts, I will show you How to use Entity Framework Code First and Database First Approach in ASP.NET Core MVC and Web API Applications. Please read our previous article, which briefly introduced [Entity Framework Core](#). Entity Framework Core can be used with .NET Core applications and .NET 4.6, and later .NET Framework Applications.

How to Install Entity Framework Core?

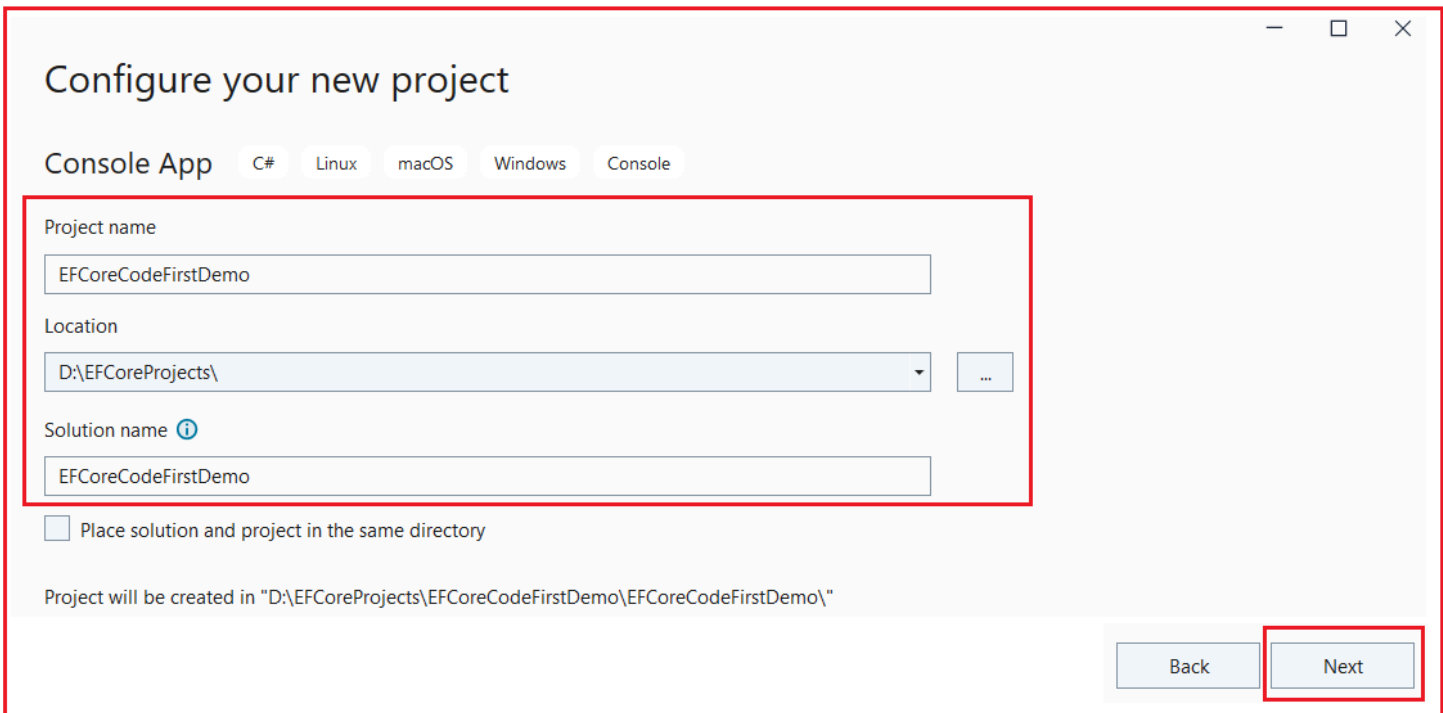
The steps to Install Entity Framework Core will be the same regardless of the type of .NET Core Application, such as a Console, Class Library, ASP.NET Core MVC, ASP.NET Core Web API, etc. Let's create a new .NET Core Console Application and then see how to Install Entity Framework Core into our project. First, open Visual Studio and click "Create a new Project," as shown in the image below.



Then select the Console App, which targets .NET Core, i.e., the Console Application, which can run on .NET on Windows, Linux, and macOS, and click on the Next button, as shown in the image below. As I am using Windows OS, I have filtered the language as C#, OS as Windows, and Application type as Console.



Once you click on the Next button, the following Configure Your New Project window will open. Here, you need to provide the Project Name (I am providing the project name as **EFCoreCodeFirstDemo**), the location (I am creating in **D:EFCoreProjects**) where you want to create the project, and the solution name (I am keeping the solution name the same as the project name) and then click on the **Next** button, as shown in the image below.



Once you click on the Next button, it will open the following Additional Information Window. Here, select the Target .NET Framework. I am selecting .NET 8 (the latest version at this moment), and I don't want to use the top-level statements, so I am checking the Do not use top-level statement checkbox and finally clicking the Create button, as shown in the image below.

Additional information

Console App

C#

Linux

macOS

Windows

Console

Framework ⓘ

.NET 8.0 (Long Term Support)

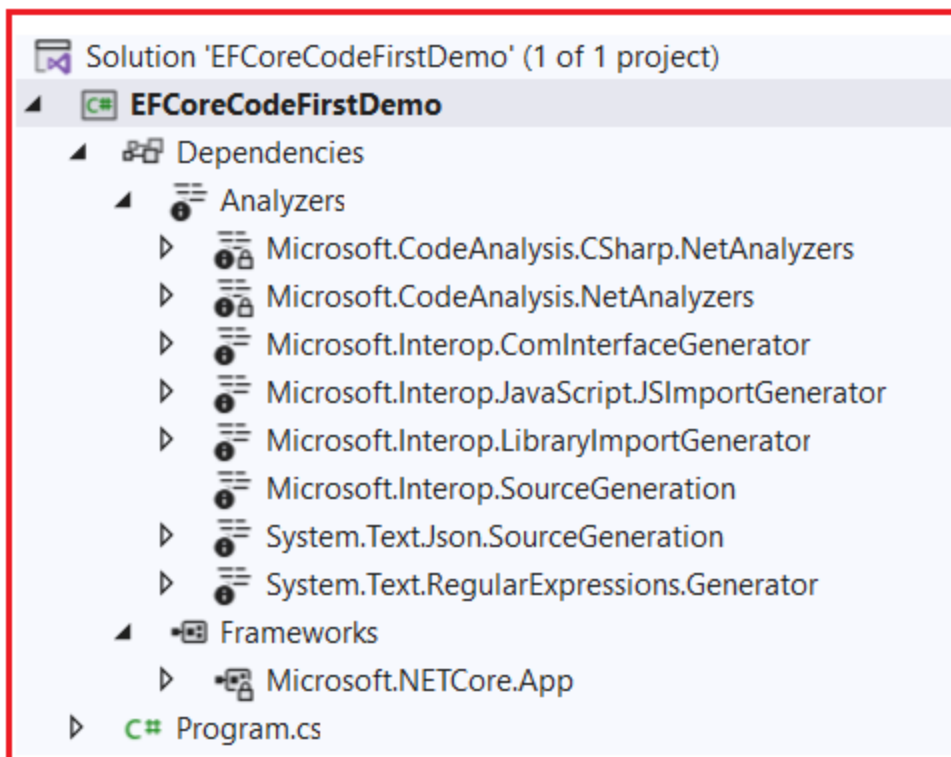
☒ Do not use top-level statements ⓘ

☐ Enable native AOT publish ⓘ

Back

Create

Once you click the Create button, it will create the Console Application using .NET 8 with the following structure.



As you can see, Entity Framework Core is not installed by default. This is because .NET Core follows a modular development approach. That means the minimum things required to develop and run an application will be provided when you create the project. The rest of the things which is required by your application can be installed from NuGet as a package. In this way, unnecessary things are removed from your project, reducing the project size and improving the application start-up performance. As we want to communicate with the database using Entity Framework Core, let's proceed and try to understand how to Install Entity Framework Core in our .NET Core Console Application.

Installing Entity Framework Core Packages:

The Entity Framework Core is not a part of the .NET Core and standard .NET framework. It is available as a NuGet Package. To use Entity Framework Core (EF Core) in .NET 8 Console Application, we need to install the following two main packages:

1. **EF Core DB Provider**
2. **EF Core Tools**

Entity Framework Core DB Provider:

The EF Core DB Provider Package is necessary because it allows EF Core to interact with the database. EF Core supports multiple databases, and you need to install the corresponding database provider for the database you want to use (e.g., SQL Server, SQLite, PostgreSQL, etc.). The following is the list of some of the popular databases and their corresponding EF Core Database Provider:

- **Microsoft SQL Server:** The NuGet Package for Microsoft SQL Server is `Microsoft.EntityFrameworkCore.SqlServer`
- **SQLite:** The NuGet Package for SQLite is `Microsoft.EntityFrameworkCore.Sqlite`
- **MySQL:** The NuGet Package for MySQL is `Pomelo.EntityFrameworkCore.MySql`
- **PostgreSQL:** The NuGet Package for PostgreSQL is `Npgsql.EntityFrameworkCore.PostgreSQL`
- **InMemory:** The NuGet Package for In-Memory Database is `Microsoft.EntityFrameworkCore.InMemory`.
- **Oracle:** The NuGet Package for Oracle Database is `Oracle.EntityFrameworkCore`
- **MongoDB:** The NuGet Package for MongoDB Non-Relational Database is `MongoDB.EntityFrameworkCore`

Without the EF Core Database Provider package, EF Core cannot perform database operations, as it doesn't know how to connect and communicate with the backend database.

Features Provided by the Package:

- **Database Connectivity:** The EF Core Database Provider package provides the necessary classes and methods to establish a connection with the database.
- **LINQ Support:** This Package also enables LINQ queries to be translated into SQL queries that the database can understand.
- **Database Migrations:** Supports database migrations to update the database schema over time.
- **CRUD Operations:** It also provides APIs (i.e., methods) for performing Create, Read, Update, and Delete operations on the database.

How to Install EF Core Database Provider:

We are going to work with the SQL Server database, so we need to install the EF Core DB Provider package for SQL Server. You can install the Package using Package Manager for Solution and Package Manager Console.

For example, please execute the following command in the Package Manager Console to install Entity Framework Core SQL Server Provider Package. You can open the Package Manager Console by following **Tools -> NuGet Package Manager -> Package Manager Console** and then execute the following command:

Install-Package Microsoft.EntityFrameworkCore.SqlServer

Note: Replace `Microsoft.EntityFrameworkCore.SqlServer` with the provider for the database you are using (e.g., `Npgsql` for PostgreSQL, `Sqlite` for SQLite).

Entity Framework Core Tools Package

The EF Core Tools package is crucial for managing the database and its schema during development. It provides tools to create migrations, update databases, and scaffold existing databases into your application.

Features Provided by the Package:

- **Migrations:** This tool allows you to create and apply migrations to keep your database schema in sync with your data model.
- **Database Updates:** Provides commands to apply migrations to your database.
- **Removing Migrations:** Remove the latest migration, allowing you to make additional changes before regenerating the migration.
- **Scaffolding:** Generates models and DbContext from an existing database schema, which is useful when working with legacy databases.

How to Install EF Core Tools:

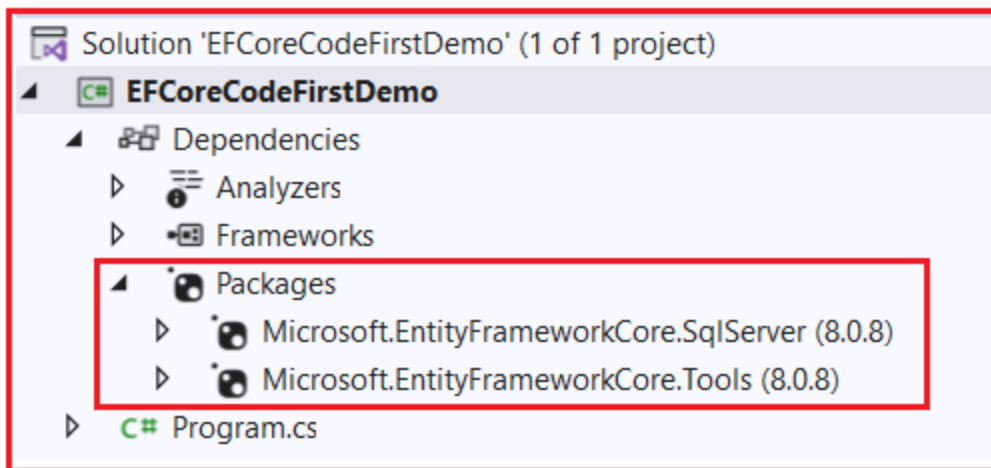
Again, you can install the EF Core Tools package using Package Manager for Solution or the Package Manager Console. Please execute the following command in Package Manage Console to install the EF Core Tools Package in your project:

Install-Package Microsoft.EntityFrameworkCore.Tools

Note: This package is typically used during development and does not need to be included in the final deployed application.

Verifying the Packages:

After successfully installing the packages, you can verify them from Solution Explorer under the Dependencies => Packages folder, as shown in the image below.



Installing these two packages ensures that our .NET 8 Console Application can effectively work with SQL Server database using Entity Framework Core. Next, we need to understand the **DbContext** class.

In the next article, I will discuss the [DbContext Class in Entity Framework Core](#) with Examples. In this article, I explain **how to install the Entity Framework Core in a .NET Core Application** using Visual Studio. I hope you enjoy this Install the Entity Framework Core in the .NET Core Application article.